Genetic Algorithm for Behaviour Tree (BT)

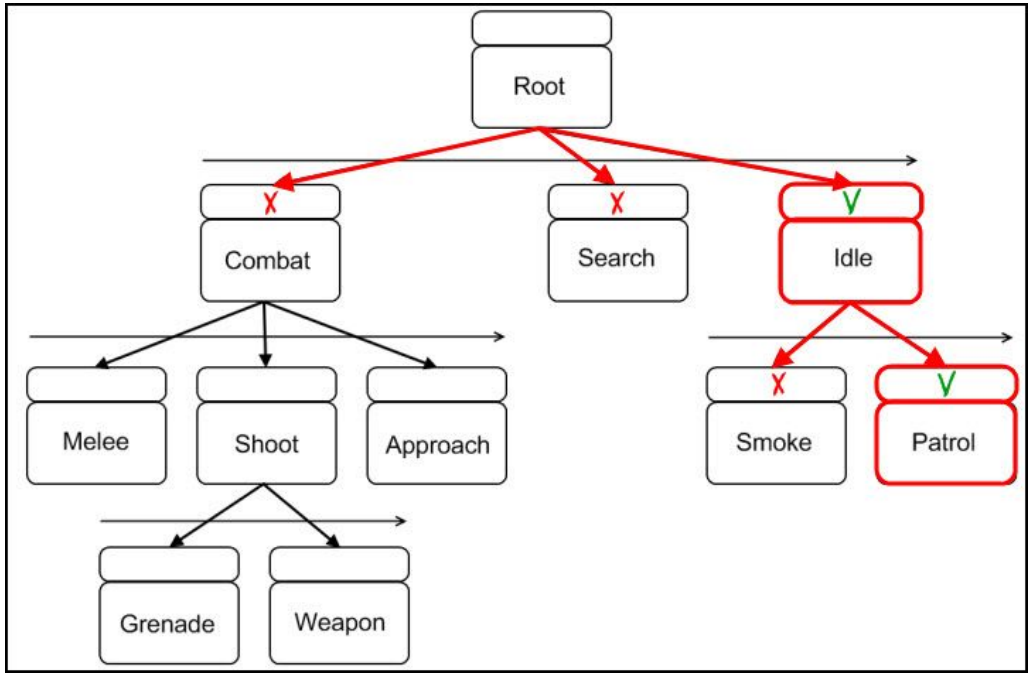
Darko Bozhinoski,
Ph.D. in Computer Science

Agenda

- Behavior Trees (BT)
- Genetic algorithm for BT
- Example. Evolution of controllers in robot swarms

BT: Game Industry

- Originated from Game Industry, as a powerful way to describe AI for “NPC”
  - Halo, Bioshock, Spore



Behaviour Tree Architecture

- Mathematical Model of Plan Execution – describe switching between a finite set of behaviours in a *modular* fashion
- Properties
  - Expressiveness
  - Accuracy
  - one-way vs. two-way control transfer (BTs are more modular than FSMs is that they use a two-way control transfer)
  - Implementation (memory-less vs. memory-full nodes)

Behaviour Tree Architecture (2)

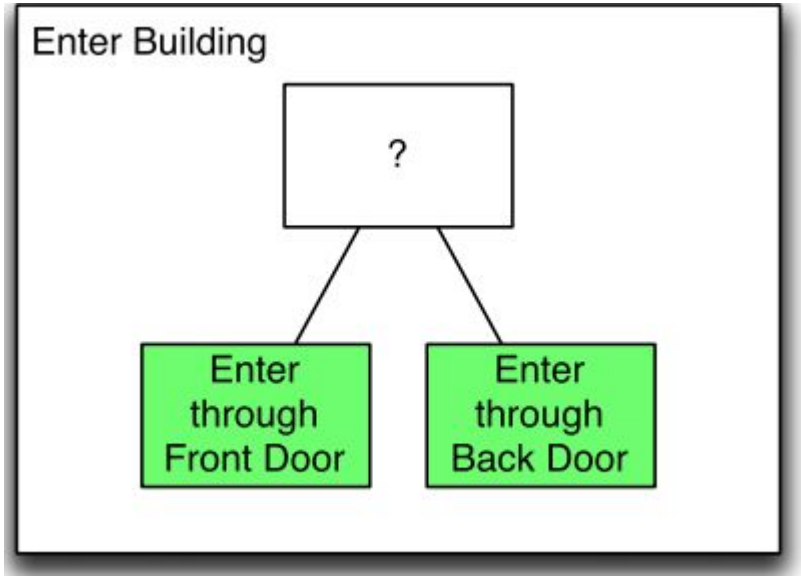
BT is directed tree in which the nodes are classified as:

- *Root, control flow nodes and execution nodes*

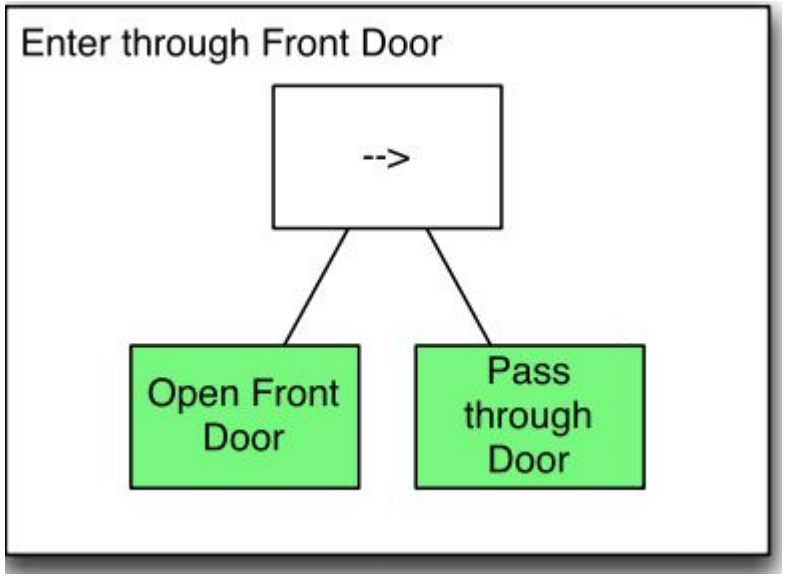
- The execution of a BT starts from the root which sends ticks with a certain frequency to its child.

Node Type	Symb.	Succeeds if	Fails if	Runs if
Root	⌀	tree S	tree F	tree R
Selector	?	1 Ch S	N Ch F	1 Ch R
Sequence	→	N Ch S	1 Ch F	1 Ch R
Parallel	⇉	$\geq S$ Ch S	$\geq F$ Ch F	otherwise
Decorator	◇	varies	varies	varies
Action n	□	$X_n(t) \in \mathsf{S}_n$	$X_n(t) \in \mathsf{F}_n$	$X_n(t) \in \mathsf{R}_n$
Condition n	○	$X_n(t) \in \mathsf{S}_n$	$X_n(t) \in \mathsf{F}_n$	never

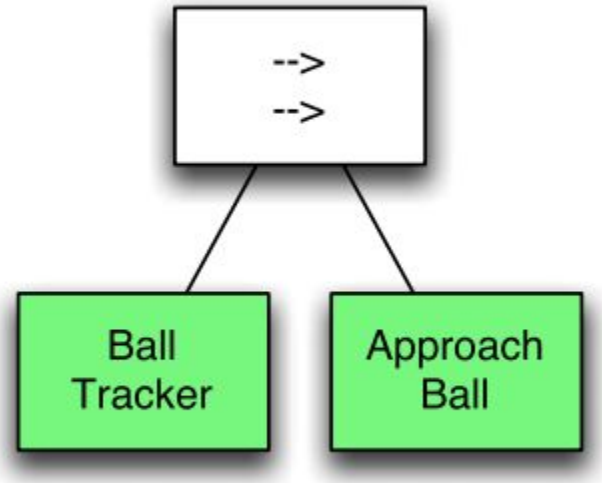
Selector


Enter Building
?
Enter through Front Door
Enter through Back Door

Sequence



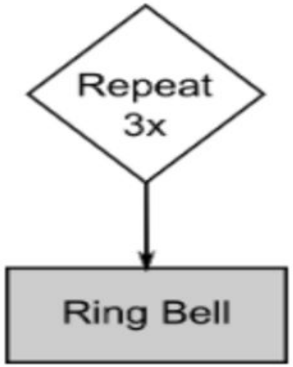
Parallel



Decorator

Decorators are special nodes that can have only a single child. The goal of the decorator is to change the behavior of the child by manipulating the returning value or changing its ticking frequency

- Repeater
- Inverter

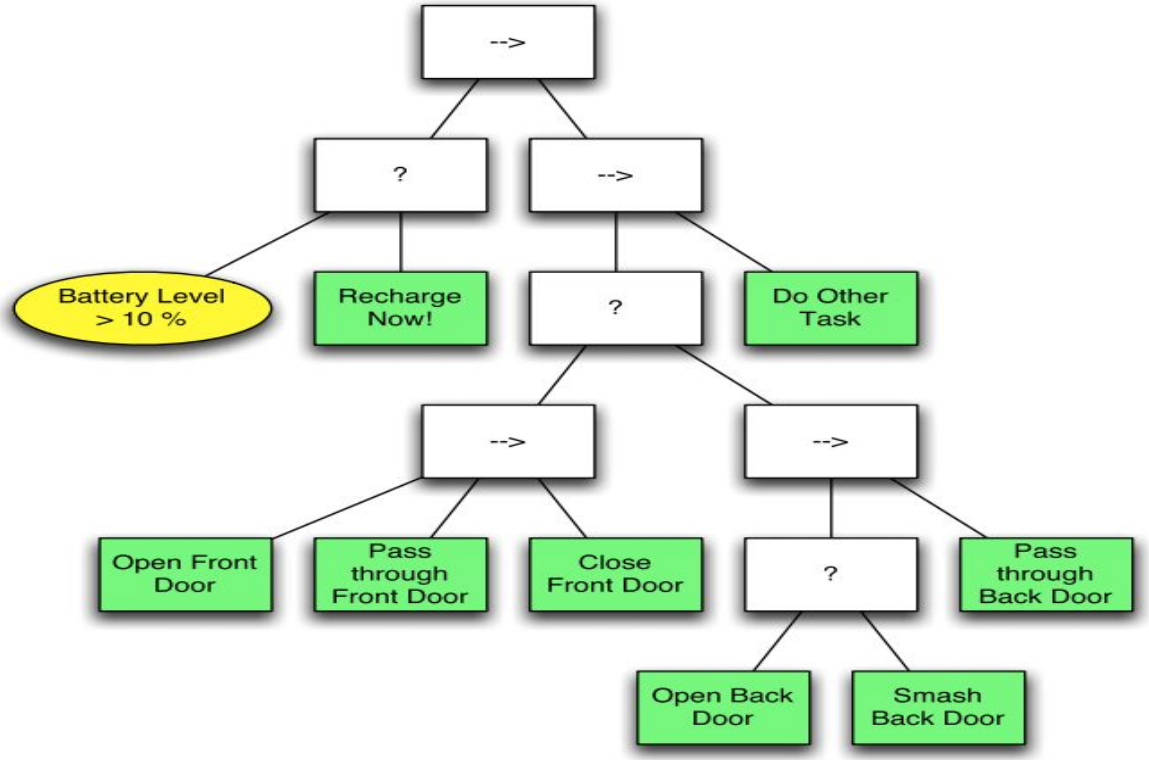


Execution Nodes

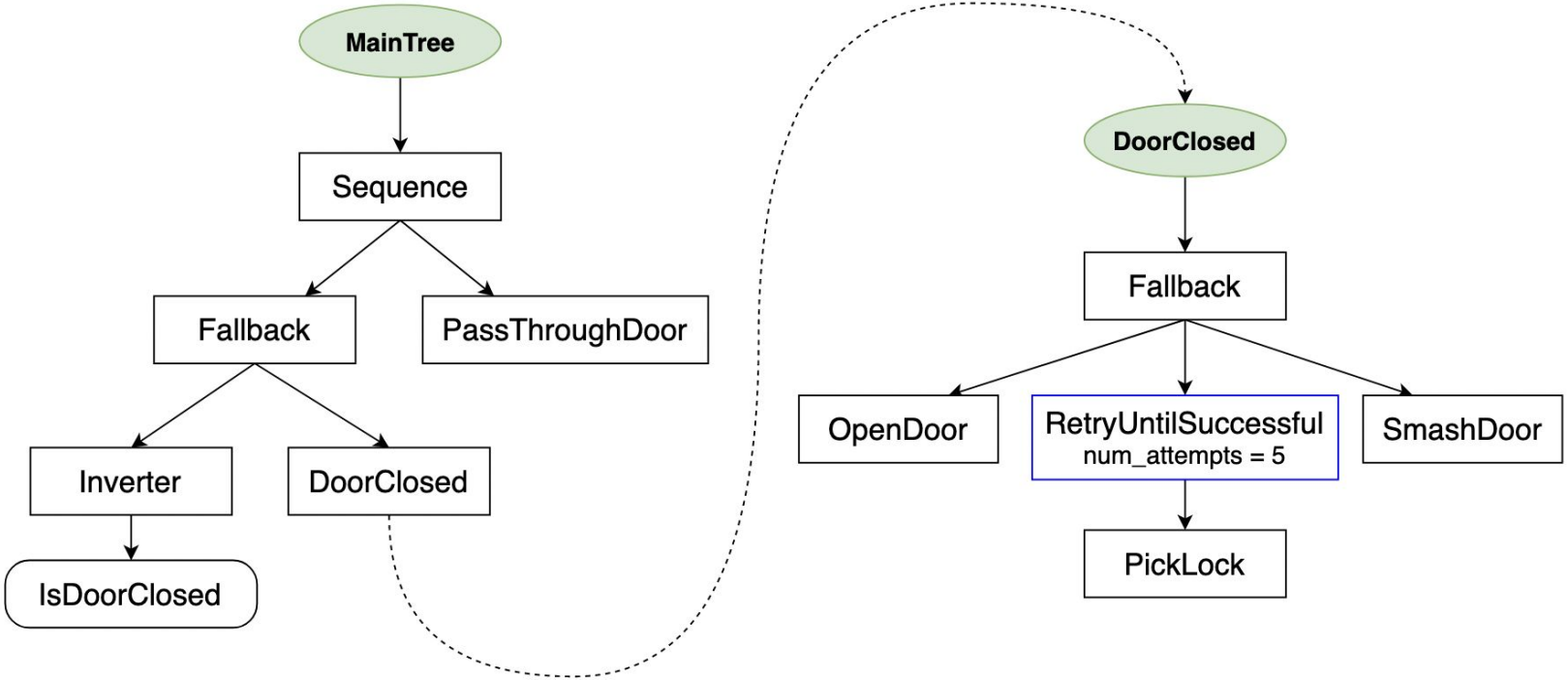
The execution nodes are the primitive building blocks of the behaviour tree. They perform some computation and return a state value (functional).

- **Condition node** checks whether a certain conditions has been meet and returns TRUE or FALSE to its parent
- **Action node** (behaviour in our case) performs computations to change the agent state and returns TRUE, FALSE or RUNNING to its parent.

Example BT


-->
?
-->
Battery Level
> 10 %
Recharge
Now!
?
Do Other
Task
-->
-->
Open Front
Door
Pass
through
Front Door
Close
Front Door
?
Pass
through
Back Door
Open Back
Door
Smash
Back Door

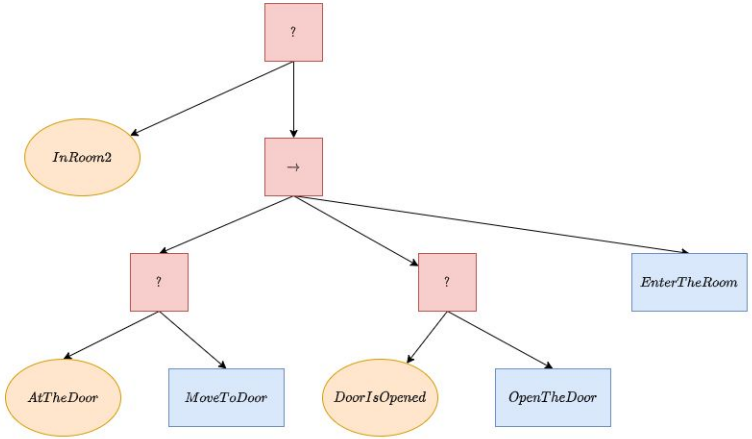
Modularity of BT

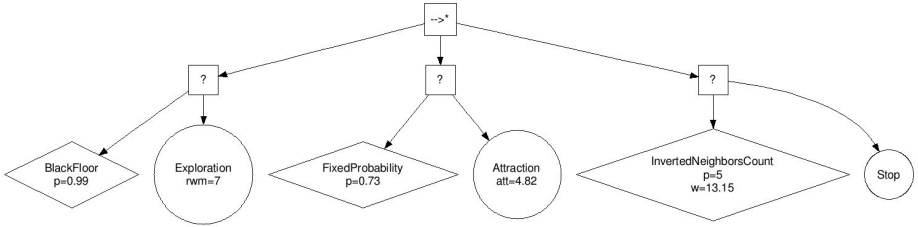


BENEFITS OF BT

- **Maintainability:** transitions in BT are defined by the structure, not by conditions inside the states. Because of this, nodes can be designed independent from each other, thus, when adding or removing new nodes (or even subtrees) in a small part of the tree, it is not necessary to change other parts of the mode.
- **Scalability:** when a BT have many nodes, it can be decomposed into small sub-trees saving the readability of the graphical model.
- **Reusability:** due to the independence of nodes in BT, the subtrees are also independent. This allows the reuse of subtrees among other trees or projects.

Other examples





Example. Design Behaviour Tree (5 min)

1. Design a Behaviour Tree for a mobile robot navigating in a supermarket.

Genetic Algorithm Pseudocode

Algorithm 6: Pseudocode of a GA.

1 Randomly generate an initial population of I individuals.
2 Calculate cost of each individual i in the population.
3 Select individuals for next generation.
4 Change individuals randomly.
5 Repeat 2-4 until criteria is achieved.

BT crossover



BT crossover

Crossover switches a subtree or a single leaf node of a BT with another BT at random. The key is that all parts of the BT can be switched, from single behaviors to large subtrees. After selection, the parents are randomly paired and a random node from each BT is chosen. The crossover can be as easy as switching two leaf nodes between two parents, but can also be used to inherit large subtrees.

BT crossover (2)

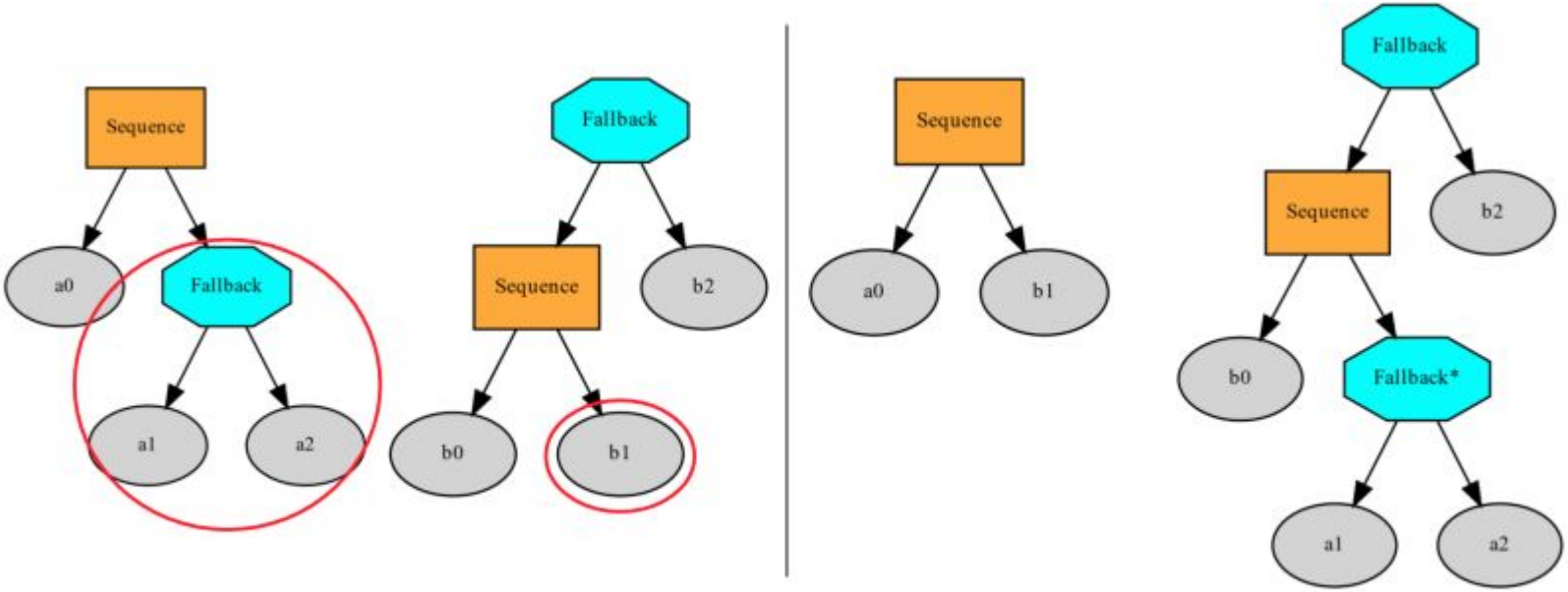


Figure 22: Result of a simple crossover. The parents before crossover can be seen to the left with red circles surrounding the crossover parts. The right part of the tree shows the offspring result.

BT mutations



BT mutations

Adding, deleting and switching a node

Adding nodes

- **Add leaf node:** Since the size of the final BT is not known, the mutation can add a random leaf node to any position of the BT.
- **Add control node:** Adding a control node to the BT requires it to have some children. The control node inherits all the remaining nodes to the right of where it is added, creating a subtree with all the nodes and subtrees of the current level. If the current position that the control node is added to does not have children to the right, a new position is chosen randomly.

Add a leaf node

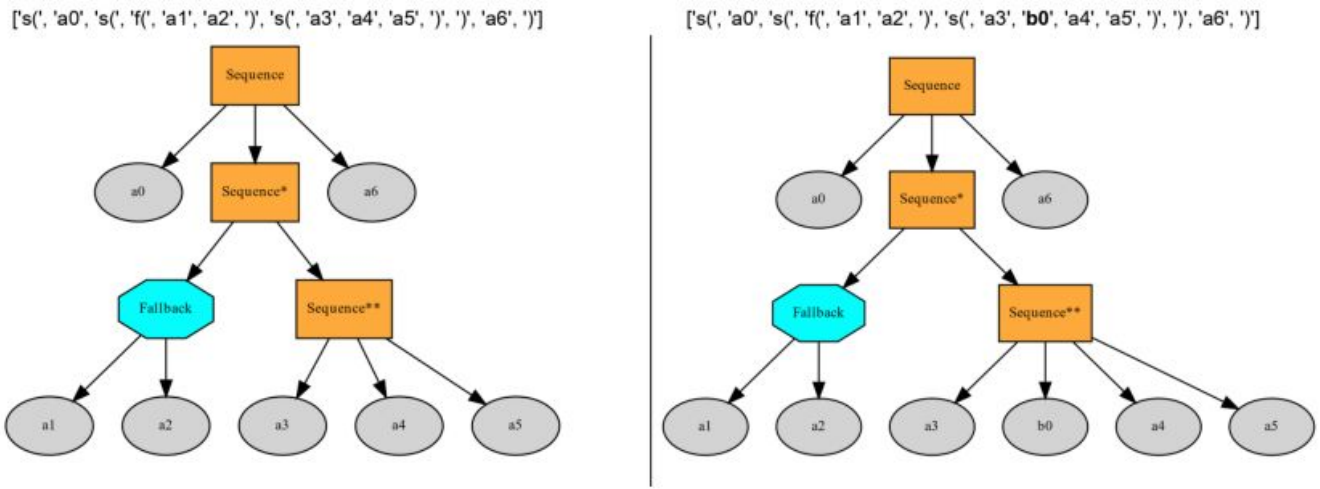


Figure 19: Add a leaf node "b0" into sequence node.

Add a control node

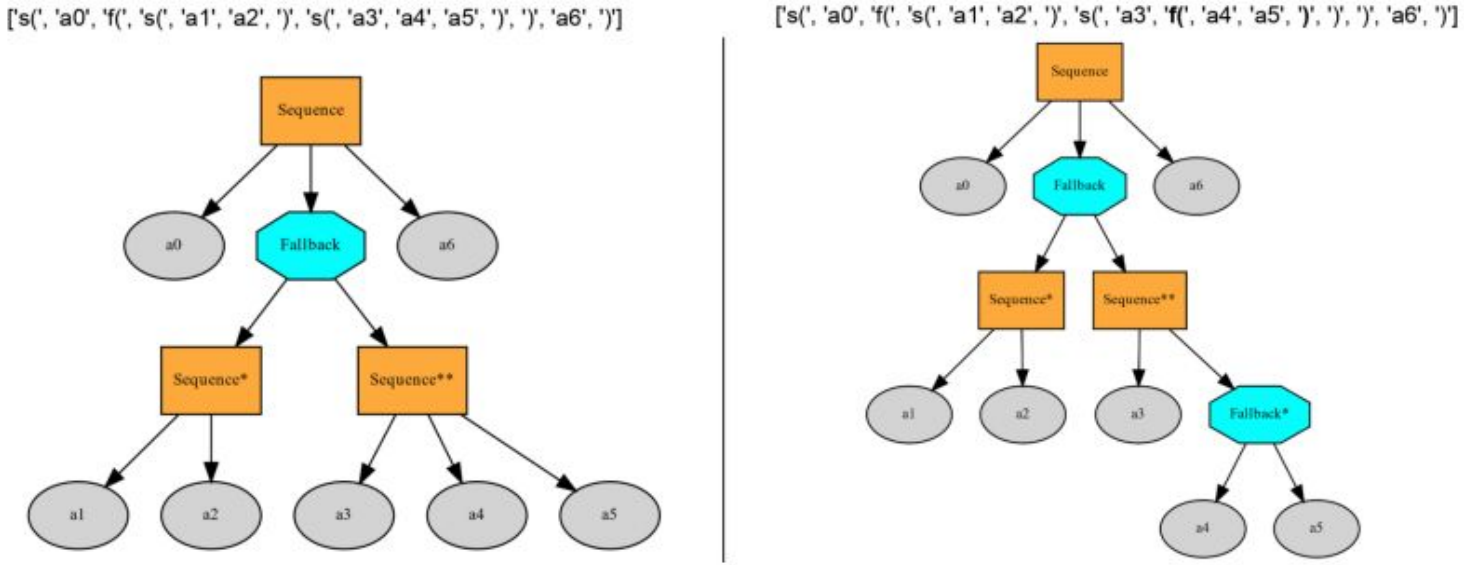
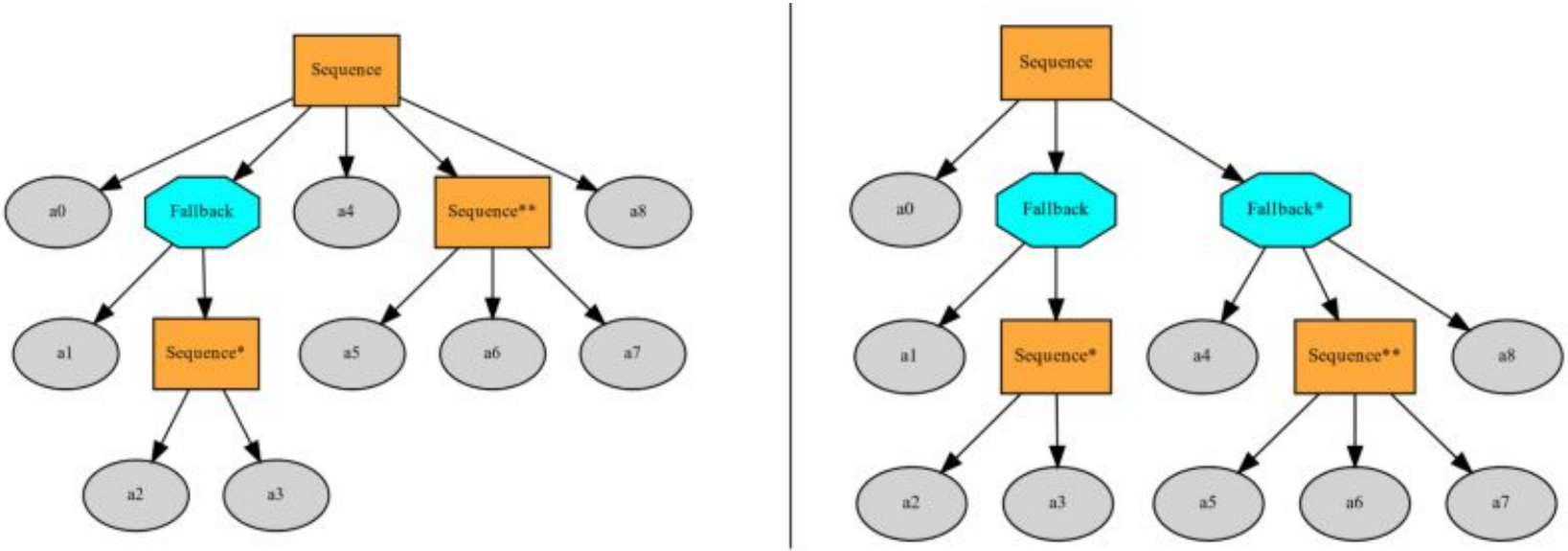


Figure 38: Add control node, which inherits the remaining leaf nodes to the right of its parent.

Add a control node (2)

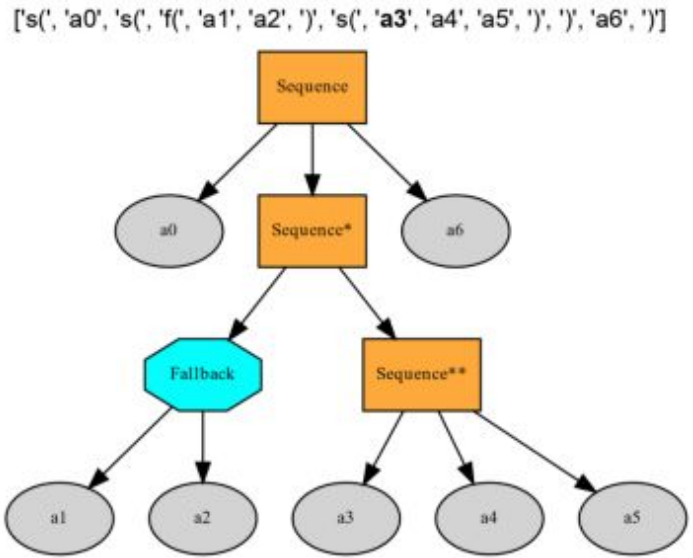


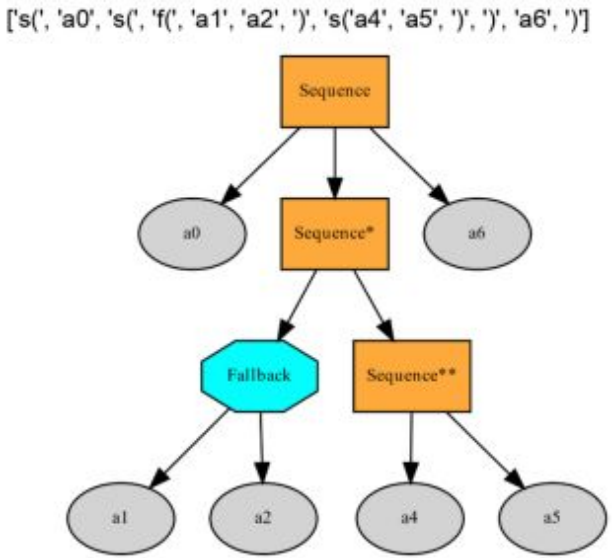
The control node inherits the remaining nodes to the right of its parent.

Deleting nodes

- Deleting a leaf node
- Deleting a control node

Delete a leaf node

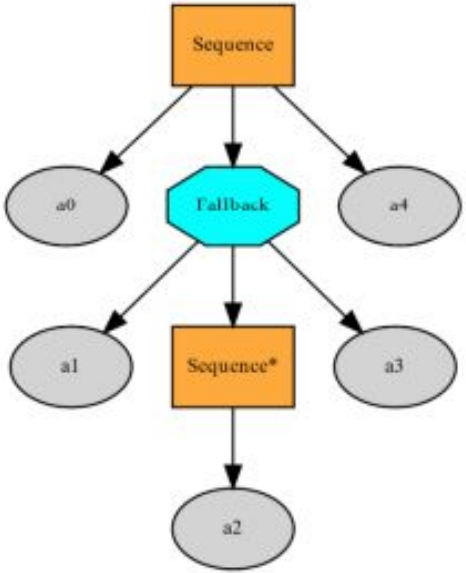




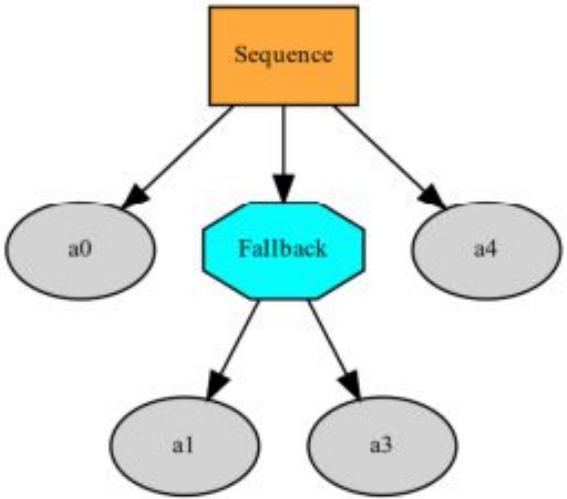
Delete leaf node “a3”

Delete a leaf node - Special case

['s(', 'a0', 'f(', 'a1', 's(', 'a2', ')', 'a3', ')', a4', ')']



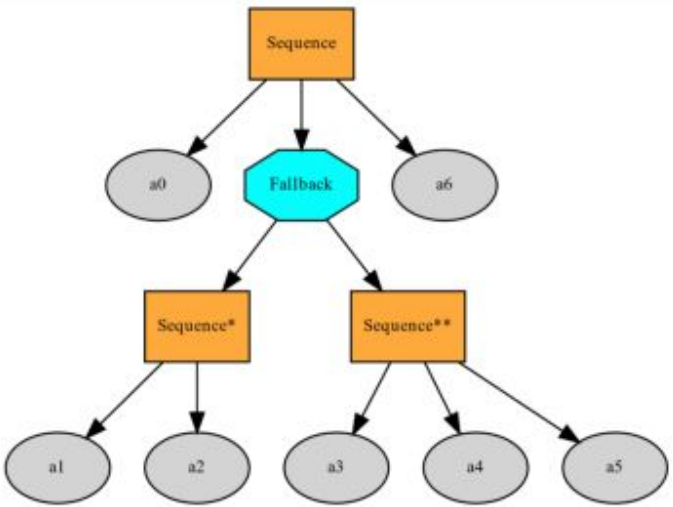
['s(', 'a0', 'f(', 'a1', 'a3', ')', a4', ')']



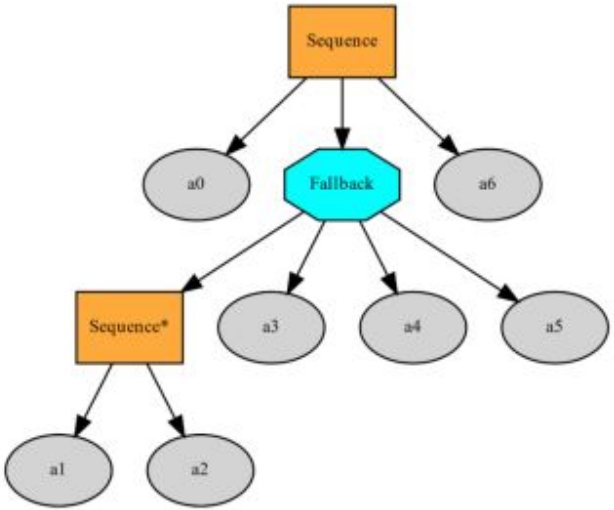
When deleting the leaf node ”a2” that is the only child of a control node, also delete the control node.

Delete control node

['s(', 'a0', 'f(', 's(', 'a1', 'a2', ')', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']



['s(', 'a0', 'f(', 's(', 'a1', 'a2', ')', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']



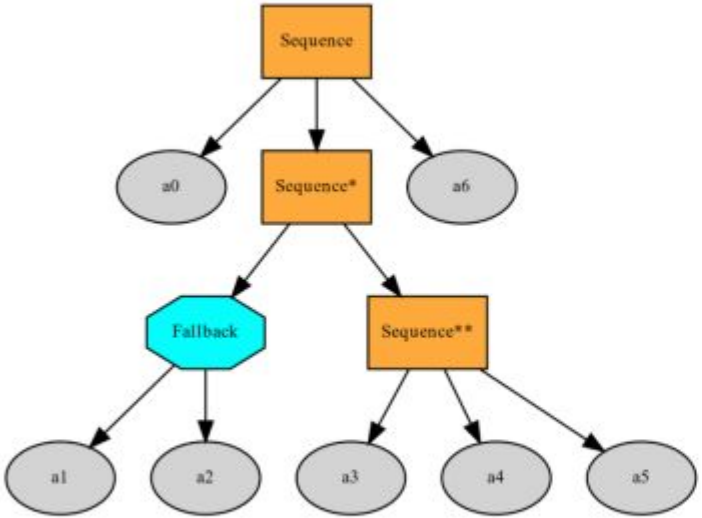
The parent keeps the control node’s children.

Switching nodes

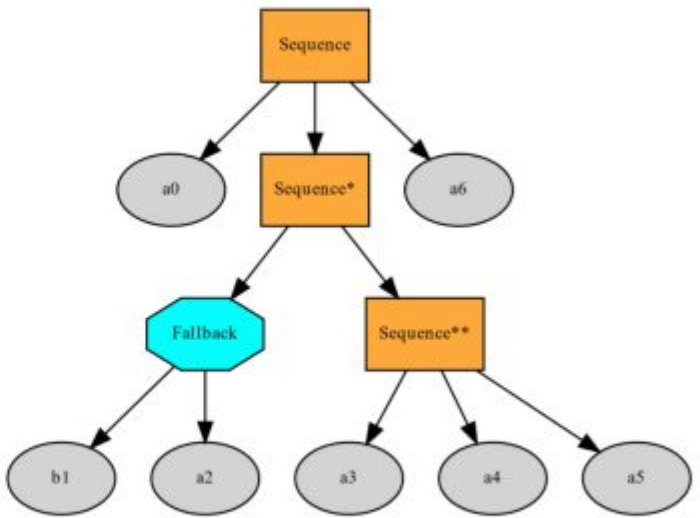
- Switch a leaf node into another leaf node
- Switch a control node into another control node
- Switch a leaf note into a control node
- Switch a control node into a leaf node

Switch a leaf node into another leaf node

['s(', 'a0', 's(', 'f(', **'a1'**, 'a2', ')', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']

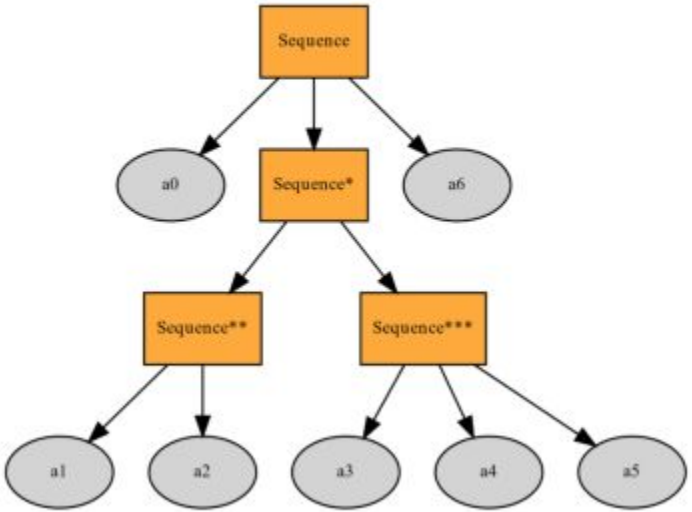


['s(', 'a0', 's(', 'f(', **'b1'**, 'a2', ')', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']

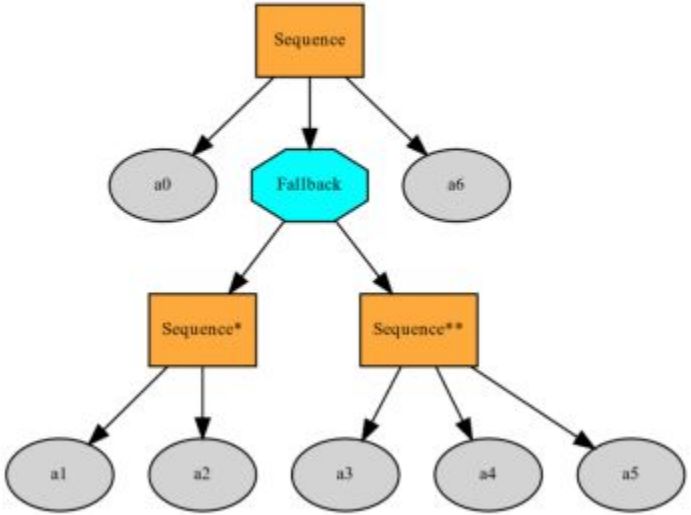


Switch a control node into another control node

['s(', 'a0', '**s(**', 's(', 'a1', 'a2', ')', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']

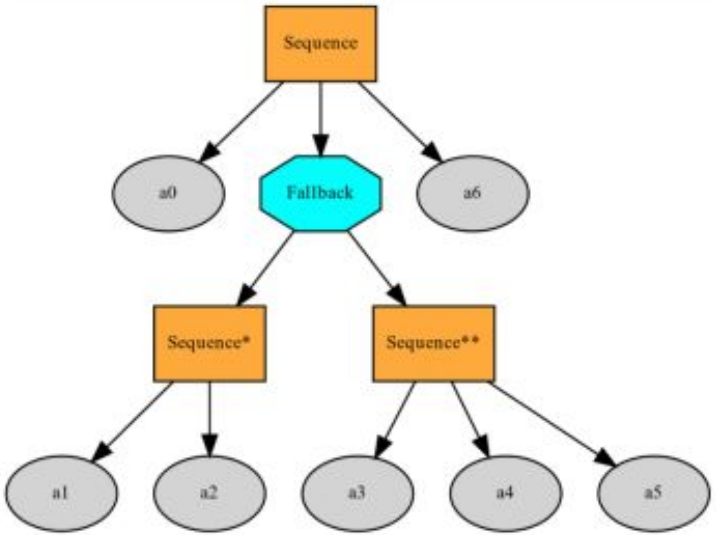


['s(', 'a0', '**f(**', 's(', 'a1', 'a2', ')', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']

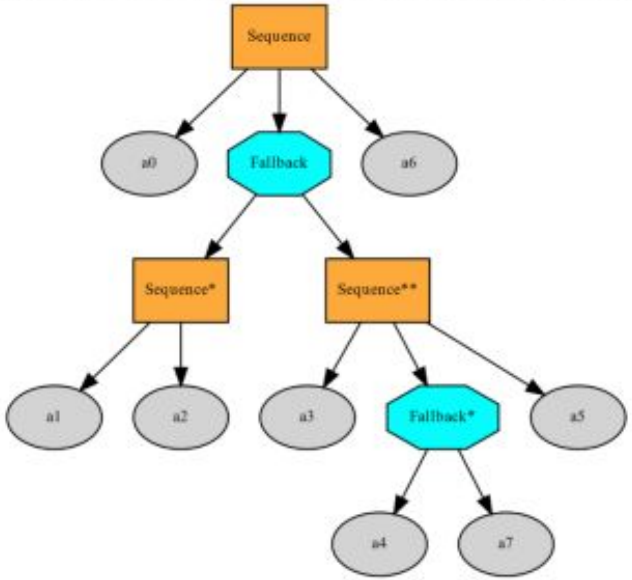


Switch a leaf node into a control node

['s(', 'a0', 'f(', 's(', 'a1', 'a2', ')', 's(', 'a3', **'a4'**, 'a5', ')', ')', 'a6', ')']



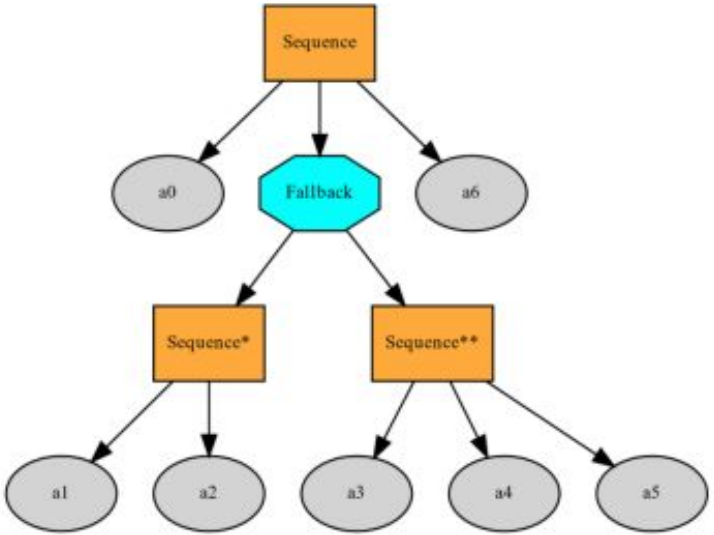
['s(', 'a0', 'f(', 's(', 'a1', 'a2', ')', 's(', 'a3', **'f('**, **'a4'**, **'a7'**, ')', 'a5', ')', ')', 'a6', ')']



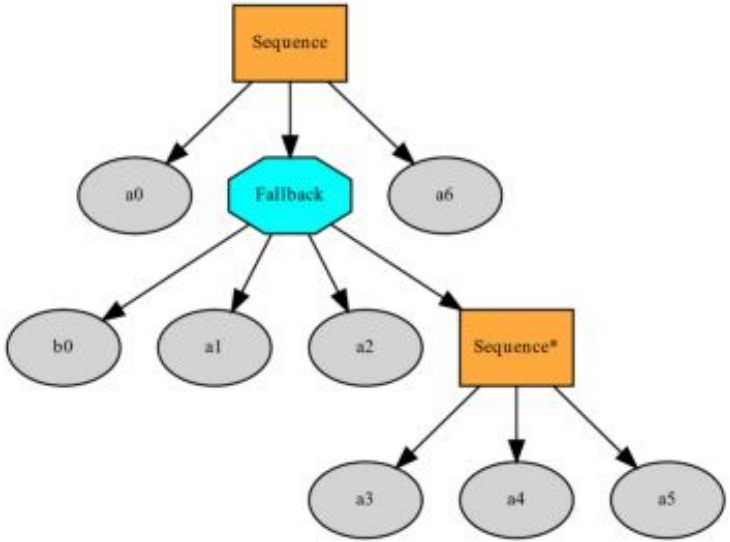
The control node keeps the mutating leaf node and adds a random behavior

Switch a control node into a leaf node

['s(', 'a0', 'f(', '**s(**', 'a1', 'a2', '**)**', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']



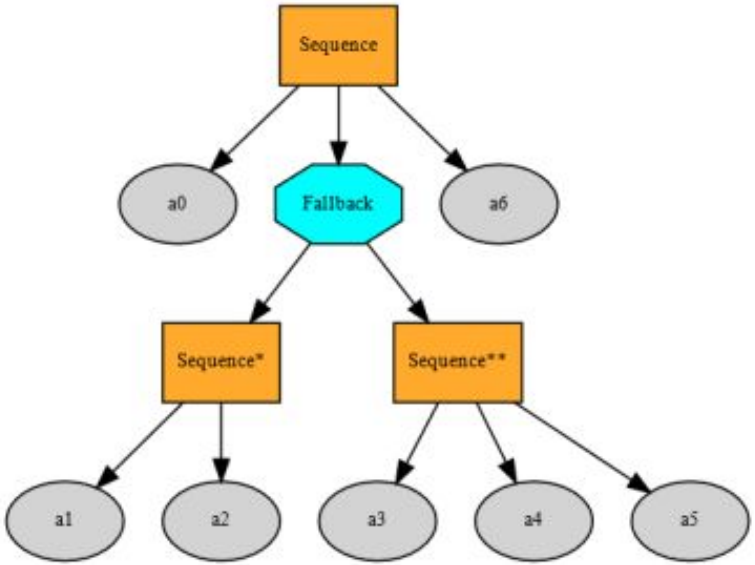
['s(', 'a0', 'f(', '**b0**', 'a1', 'a2', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']



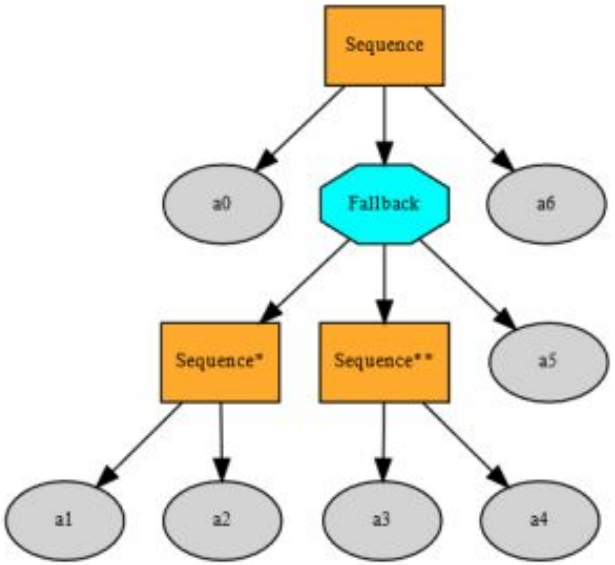
The bottom left control node is switched to a leaf node ”b0”.

Modifying the tree structure

['s(', 'a0', 'f(', 's(', 'a1', 'a2', ')', 's(', 'a3', 'a4', 'a5', ')', ')', 'a6', ')']

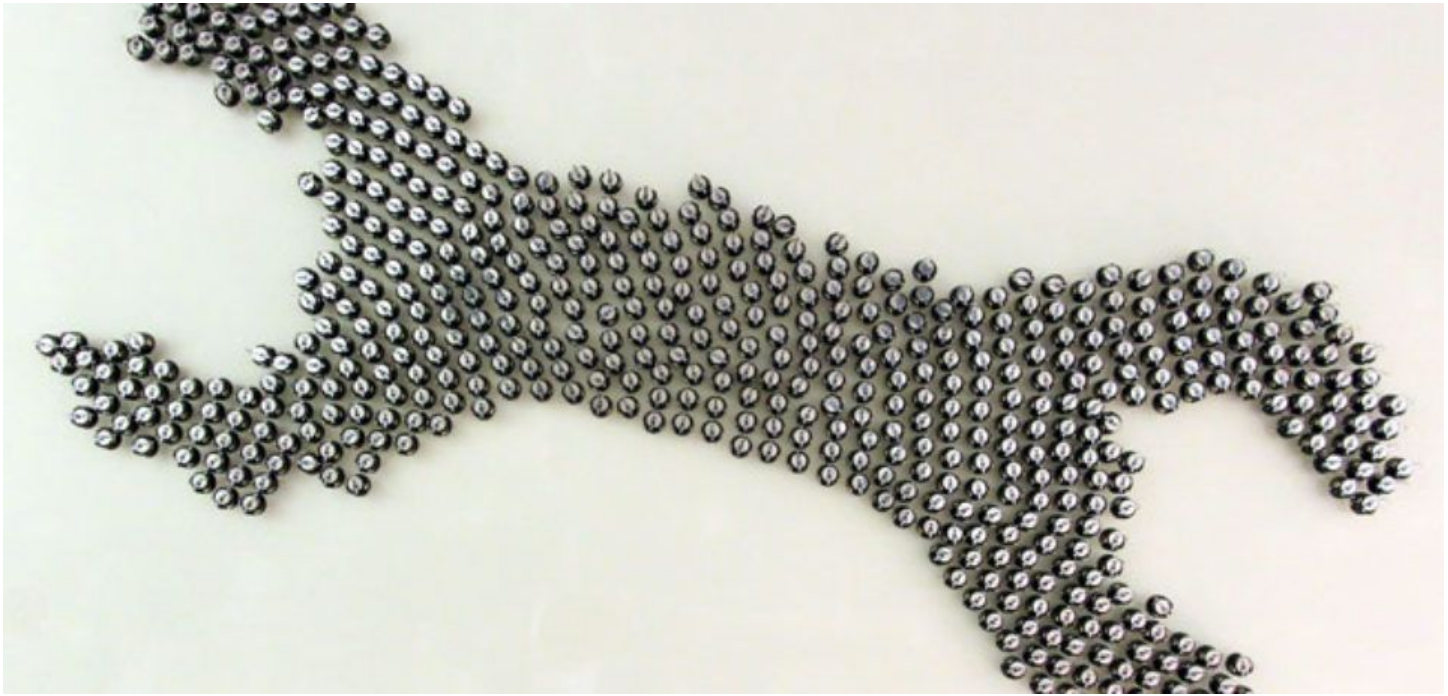


['s(', 'a0', 'f(', 's(', 'a1', 'a2', ')', 's(', 'a3', 'a4', ')', 'a5', ')', 'a6', ')']



Closing control nodes of the BT.

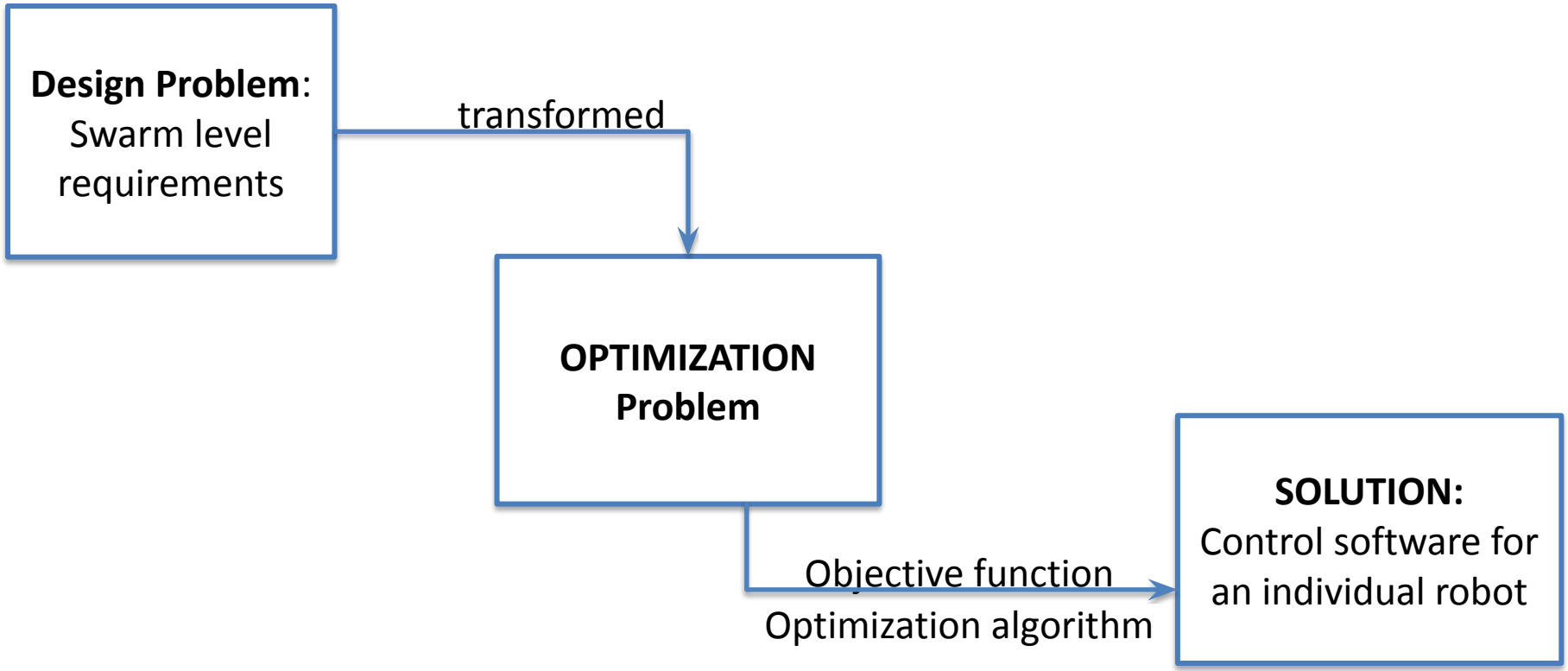
Robot Swarms



Robot Swarm(2)

- Robot swarm operates in a self-organized and distributed manner
- Swarm collective behavior
  - Robot-robot interaction
  - Robot-environment interaction

Automatic Design Approach



Automatic Design Approach (2)

- Evolutionary swarm robotics
  - control software of each individual robot (Behaviour Tree, FSM)
  - evolutionary algorithm optimizes a mission-specific objective function (Fitness function)

Evolution of controllers in robot swarms using GA

Springer Link



Distributed Autonomous Robotic Systems pp 487–501 | Cite as

Evolving Behaviour Trees for Swarm Robotics

Simon Jones ✉, Matthew Studley, Sabine Hauert & Alan Winfield

Chapter | First Online: 14 March 2018

2272 Accesses | **21** Citations | **7** Altmetric

Part of the Springer Proceedings in Advanced Robotics book series (SPAR,volume 6)

Abstract

Controllers for swarms of robots are hard to design as swarm behaviour emerges from their interaction, and so controllers are often evolved. However, these evolved controllers are often difficult to understand, limiting our ability to predict swarm behaviour. We suggest behaviour trees are a good control architecture for swarm robotics, as they are comprehensible and promote modular reuse. We design a foraging task for kilobots and evolve a behaviour tree capable of performing that task, both in simulation and reality, and show the controller is

Down
Down
Sectio
View in a

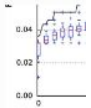
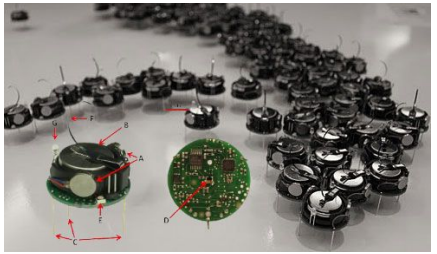


Fig. 4



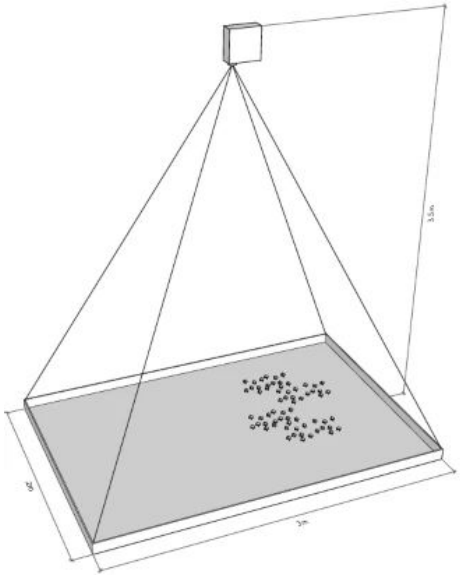
Kilobot Foraging Mission

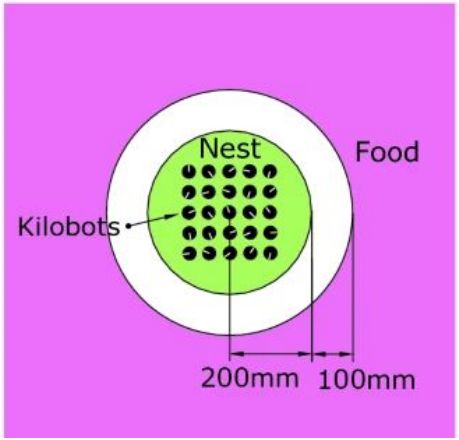
- Design of a behaviour tree controller architecture for a swarm of kilobots. Kilobot Foraging mission: https://www.youtube.com/watch?v=kioZR99hnU4



- Automatically evolve behaviour trees in simulation to enable the swarm to perform a collective foraging task.
- The fittest behaviour tree is then evaluated in a swarm of real robots and analysed

Experimental setup





Foraging mission explanation: At the centre of the arena is a circular ***nest* region**. Surrounding this is a gap, then beyond that is the ***food*** region. A kilobot which moves into the food region is regarded as having picked up an item of food, a kilobot which is carrying an item of food that enters the nest region is regarded as depositing the food in the nest.

Fitness Function

The fitness of the swarm is related to the total amount of food returned to the nest within the test time. The maximum possible number of food items depends on the starting spatial distribution of the kilobots.

Let $food_{max}$ be the maximum food items, t_{test} be the test time, v_{avg} be the average linear velocity of the kilobots, n be the number of kilobots, fn_{dist} be the shortest (radial) distance between the food and nest regions:

$$food_{max} = \frac{n \cdot v_{avg} \cdot t_{test}}{2 \cdot fn_{dist}}$$

Fitness Function (2)

Let $food_{max}$ be the maximum food items, t_{test} be the test time, v_{avg} be the average linear velocity of the kilobots, n be the number of kilobots, fn_{dist} be the shortest (radial) distance between the food and nest regions:

$$food_{max} = \frac{n \cdot v_{avg} \cdot t_{test}}{2 \cdot fn_{dist}}$$

We normalise the actual collected food items within the time of the test to give a fitness value

$$f = k \cdot \frac{food_{collected}}{food_{max}}$$

Where k is a derating factor used to exert selection pressure towards smaller behaviour trees

BT Controller

- To control a robot with a behaviour tree, they define the interface between the behaviour tree action nodes and the robot, and the action nodes that act on the interface. This interface is known as the blackboard.
- There is a trade-off between the capabilities that we choose to hard code and those that we hope will evolve in the BT.

BT Actions

Table 1 Behaviour tree blackboard, defining interface between the behaviour tree and the robot

Index	Name	Access	Description
0	$motors$	W	0 = off, 1= left turn, 2 = right turn, 3 = forward
1	$scratchpad$	RW	Arbitrary state storage
2	$send_signal$	RW	>0.5 = Send a signal flag
3	$received_signal$	R	1 = A signal flag has been received
4	$detected_food$	R	1 = Light sensor showing food region
5	$carrying_food$	R	1 = Carrying food
6	$density$	R	Density of kilobots in local region
7	$\Delta density$	R	Change in density
8	$\Delta dist_{food}$	R	Change in distance to food
9	$\Delta dist_{nest}$	R	Change in distance to nest

Evolution procedure

- The population of $npop$ is evaluated for fitness by running 10 simulations for each individual, each simulation with a different starting configuration. The starting position is always a 5×5 grid with 50 mm spacing in the centre of the nest region, but the orientation is randomly chosen from interval $(-\pi,\pi)$ radians. The simulation runs for 300 simulated seconds
- An elite of $nelite$ is transferred unchanged to the next generation.
- The remainder are chosen by tournament selection with size $tsize$.
- A tree crossover operator is applied with probability $pxover$ to all pairs of non-elite, then three different mutation operators are applied to the non-elite individuals.
    - Firstly, with probability $pmutu$, a node in the tree is selected at random and the subtree at that point is replaced with a randomly generated one.
    - Next, with probability $pmuts$, a branch is chosen randomly and replaced with one of its terminals.
    - Next, with probability $pmutn$ a node is picked at random and replaced with another node with the same argument types.
    - Lastly, with probability $pmute$, a constant is picked randomly and its value changed

Table 3 Parameters for a single evolutionary run

From: Evolving Behaviour Trees for Swarm Robotics

Parameter	Value	Description
n_{gen}	200	Generations
t_{test}	300	Test length in seconds
n_{pop}	25	Population
n_{elite}	3	Elite
t_{size}	3	Tournament size
p_{xover}	0.8	Crossover probability
p_{mutu}	0.05	Probability of subtree replacement
p_{muts}	0.1	Probability of subtree shrink
p_{mutn}	0.5	Probability of node replacement
p_{mute}	0.5	Probability of ephemeral constant replacement

Table 4 Individuals from top five lineages and their usage of the blackboard and behaviour tree constructs. All individuals use at least the forward and one other of the motor action nodes. Usage is after redundant or unreachable nodes have been removed

From: Evolving Behaviour Trees for Swarm Robotics

Rank	Fitness	Blackboard entry									BT Nodes					
		1	2	3	4	5	6	7	8	9	SEQ	SEL	PROB	REPEAT	IF	SET
1	0.104				x				x	x	x	x		x	x	
2	0.0873		x	x	x						x	x	x		x	x
3	0.0853				x	x		x		x	x		x		x	x
4	0.0723	x	x		x	x	x	x	x	x	x	x	x	x	x	x
5	0.0710				x		x		x		x	x	x		x	

```
selm3(
  seqm2(
    ifgevar(send_signal, detected_food),
    mf()),
  seqm3(
    seqm3(
      ml(),
      ifgevar(Δdist_food, scratchpad),
      mf()),
    ifgevar(Δdist_food, scratchpad),
    seqm2(
      seqm3(
        seqm3(
          ml(),
          ifgevar(Δdist_nest, Δdist_nest),
          mf(),
        ifgevar(Δdist_food, send_signal),
        mf()),
      mf()))),
  seqm3(
    ml(),
    repeat(5,
      ifltcon(Δdist_nest, −0.058530)),
    ml()))
```

```
selm3(
  seqm2(                          Move forward until in food
    ifge(0, detected_food),
    mf()),
  seqm8(                          Turn and forward until out of food
    ml(),
    ifge(Δdist_food, 0),
    mf(),
    ml(),
    mf(),
    ifge(Δdist_food, 0),
    mf(),
    mf()),
  seqm3(
    ml(),
    repeat(5,
      iflt(Δdist_nest, −0.058530)),
    ml()))
```

Fig. 5 Fittest behaviour tree. Left shows the code as evolved. Right shows the code with redundant lines removed by hand, the `seqm` nodes condensed, and conditionals simplified. Boxes highlight the three functional clauses


Q & A